

EETS 7302 TCP/IP Network Administration

Session 11

Linux Admin – part2:

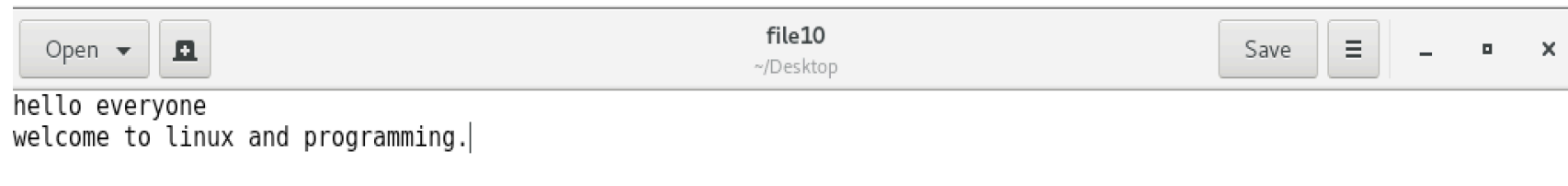
Text editors: gedit

- Our graphical text editor(gedit):
- Now it's time to learn how to view and edit text files.
- gedit is the default text editor for the Gnome Desktop Environment
- example (Viewing a file)
- If you want to view an existing text file named file10
 “gedit file10”

Text editors

- Will open up file10 for you and you can view it or edit it.

```
[smu_user@localhost Desktop]$ gedit file10
```

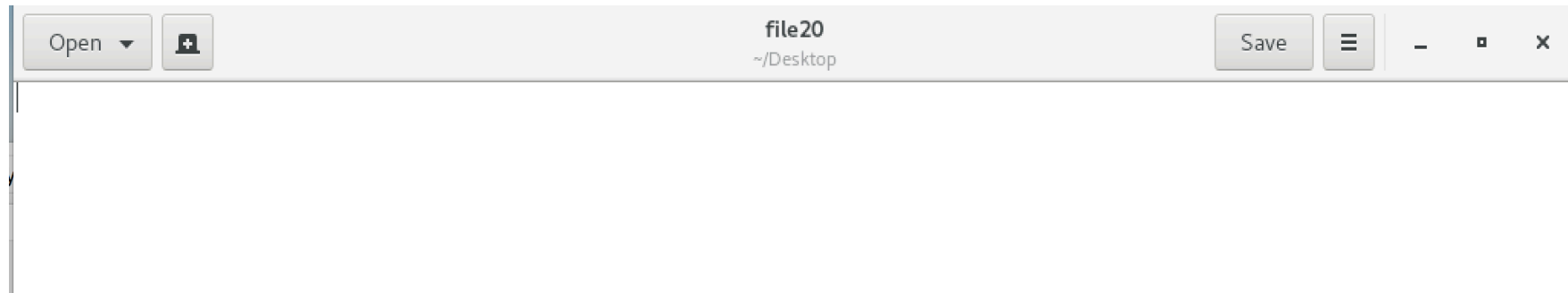


Text editors

- Example (Creating a file)
- If you want to create a new text file named “file20”
 - “gedit file20”
- Will create file20 and you can start adding text to it.
- gedit is very simple to use and is also pretty self explanatory

Text editors

```
[smu_user@localhost Desktop]$ gedit file20
```



Text editors: nano

- nano:
- Nano is a command line editor it's not a graphical editor.
- Syntax for to use nano is write "nano filename"
- Your file will open up in your terminal.
- You'll see a power symbol^ it means press control.

Nano editor

```
GNU nano 2.3.1      File: file10

hello everyone
welcome to linux and programming.

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Page  ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is   ^V Next Page  ^U UnCut Text ^T To Spell
```

vi editor

- The **vi** editor is installed on almost every Unix.
- Linux will very often install **vim** (**vi improved**) which is similar.
- Every admin should know **vi(m)**, because it is an easy tool to solve problems.
- The **vi** editor is not intuitive, but once you get to know it, **vi** becomes a very powerful application.
- Most Linux distributions will include the **vimtutor**.

vi editor

- vi has the concepts of mode and it can be used as editing shell commands.

vi editor

- The vi editor starts in **command mode**. In command mode, you can type commands. Some commands will bring you into **insert mode**. In insert mode, you can type text. The **escape key** will return you to command mode.

key	action
Esc	set vi(m) in command mode.
i,a	To set insert mode
:	Line mode

vi editor

- vi insert mode
 - lowercase “i” insert at the cursor position
 - Uppercase I insert at the beginning of the line
 - Lowercase a append after the cursor position
 - A append at the end of the line

vi editor

- Line mode
 - :w writes(saves) the file
 - :w! Forces the file to be saved
 - :q quit
 - :q! Quit without saving changes
 - :wq! Write and quit
 - :x same as :wq

vi editor

- Line mode can also be use for navigation

command	action
:n	positions the cursor at line n
:\$	positions the cursor on the last line
:set nu	turn on line numbering
:set non/nonu	turn off line numbering
:help [subcommand]	:help [subcommand] get help

vi editor

- **undo and repeat (u .)**
- When in command mode, you can undo your mistakes with u. You can do your mistakes twice with . (in other words, the . will repeat your last command).

command	action
u	Undo last action
.	Repeat the last action

- **cut, copy and paste a line (dd yy p P)**

command	action
dd	cut the current line
yy	(yank yank) copy the current line
p	paste after the current line
P	paste before the current line

- When in command mode, dd will cut the current line. yy will copy the current line. You can paste the last copied or cut line after (p) or before (P) the current line.

Text editors: emacs

- It's a powerful editor
- Some people prefer vi, other emacs
- Either are great editors.
- Use what feels comfortable to you.
- Emacs syntax = **emacs [file] Editfile**
 - C-<char> Ctrl while pressing <char>.
 - M-<char> "Meta" key (alt key) while pressing <char>
 - M-<char> Esc, then type <char>

- Emacs commands

commands	description
C-h	Help
C-x C-c	Exit
C-x C-s	Save the file
C-h t	Built-in tutorial
C-h k <key>	Describe key

- Emacs Navigation:

commands	description
C-p	Previous line
C-n	Next line
C-b	Backward one character
C-f	Forward one character
M-f	Forward one word
M-b	Backward one word
C-a	Go to the beginning of the line
C-e	Go to the end of the line
M - <	Go to the beginning of the file
M - >	Go to the end of the file

- Emacs – deleting text:

commands	description
C - d	Delete a character
M - d	Delete a word

- Emacs – copying, pasting and undo:

commands	description
C - k	Kill (cut)
C - y	Yank(paste)
C - x u	Undo

- Emacs – search options

options	description
C - s	Start a forward search
C - r	Start a reverse search

- Emacs - Repeating commands

command	description
C - u N <command>	Repeat <command> N times

Managing ownership

- For configuring permissions, understanding ownership of files and directories is vital.
- Every file and directory has two owners: a user and group. These are set when it is created. While creation, the user who creates the file becomes owner and the primary group of that user becomes the group owner.
- To access file or directory, shell need to check the ownership.

Managing ownership(cont'd)

- The permissions are checked in the following sequence by the shell:
 - If you are the owner of file or directory permission is directly granted and no further check is done.
 - If you are not user, the shell will check whether you are the member of the group associated. If you are the group member, then the permissions are granted according to group permissions.
 - Finally if you are neither user nor the member of that group, you will get the permissions of others.

Managing ownership(cont'd)

- To check the permissions user **ls -l** command. Below is the example where the user has full permission while groups and others don't have any permissions.

```
[vagrant@localhost home]$ ls -l
total 4
drwx-----. 2 bob      bob      62 Nov 12 01:44 bob
drwx-----. 2 chi      chi      62 Nov 12 01:44 chi
drwx-----. 2 rob      rob      62 Nov 12 01:44 rob
drwx-----. 2 test     test     83 Nov  2 13:20 test
drwx-----. 7 vagrant  vagrant 4096 Nov  2 12:59 vagrant
```

Managing ownership(cont'd)

- The find argument will be helpful to list the files with the user owner and group owner.
- Command: `find / -user username`
In this command the files with the owner as username will be listed. To check the files owned by bob we will use, `find / -user bob`.
- However we can manipulate the ownership of files and directories with the privileges we have.

Managing ownership(cont'd)

- To change the user ownership the command is: **chown who what**. Here 'who' is the user we want and 'what' is file or directory. To change recursively user **-R** option.

```
[test@localhost ~]$ ls -l
total 0
-rw-rw-r--. 1 test test 0 Nov 12 02:18 abc.txt
-rw-rw-r--. 1 test test 0 Nov 12 02:18 script.py
```

Managing ownership(cont'd)

- For example to change the user of the file script.py to bob we will use **chown bob script.py**

```
[root@localhost test]# chown bob script.py
[root@localhost test]# ls -l
total 0
-rw-rw-r--. 1 test test 0 Nov 12 02:18 abc.txt
-rw-rw-r--. 1 bob  test 0 Nov 12 02:18 script.py
```

- We can also change the owner of the group. It can be changed using chown or chgrp command. To use chown, use . Or : in front of group name.
ex. **chown .sales /home/sales.**

Managing ownership(cont'd)

- In this example we change the group owner of directory /home/sales to group name sales. Alternatively we can use the command:
`chown :sales /home/sales`
- Another way is to use chgrp command.
for ex, `chgrp sales /home/sales` considering the above example.
- We can use `-R` to change recursively.

Managing ownership(cont'd)

- Where does the default file permissions come from?
 - The user who creates the file becomes the user and the primary group of the associated user is defined in `/etc/passwd` file.
 - From this users primary group is selected and that group becomes the group owner for that file.
 - In case of user is member of multiple group we can change the user primary group using `newgrp` command.

Managing permissions

- The permissions for the files and directories are read, write and execute for user, group and others. If the access is restricted then it is represented by -.
- Each permission are represented by a single letter:
 - r – read permissions
 - w – write permissions
 - x – execute permissions

Managing permissions(cont'd)

- Read permission to a file means you have right to open a file for reading.
- Read for the directory means you can list the contents of the directory.
- However read permission for directory doesn't mean you can read a file from the directory.
- Write permission on a file allows you to modify the contents of the file. These permissions on directory allows you to create and remove new subdirectories.

Managing permissions(cont'd)

- The execute permissions allows you to execute a file or run a program.
- execute permission on a directory means user can use **cd** command to enter in directory.

Managing permissions(cont'd)

- Overview of permissions on files and directories.

Permission	Files	Directories
Read	Open a file	List the contents of file
Write	Change contents of file	Create and delete files and modify the permissions of file
Execute	Run a program file	Change to the directory

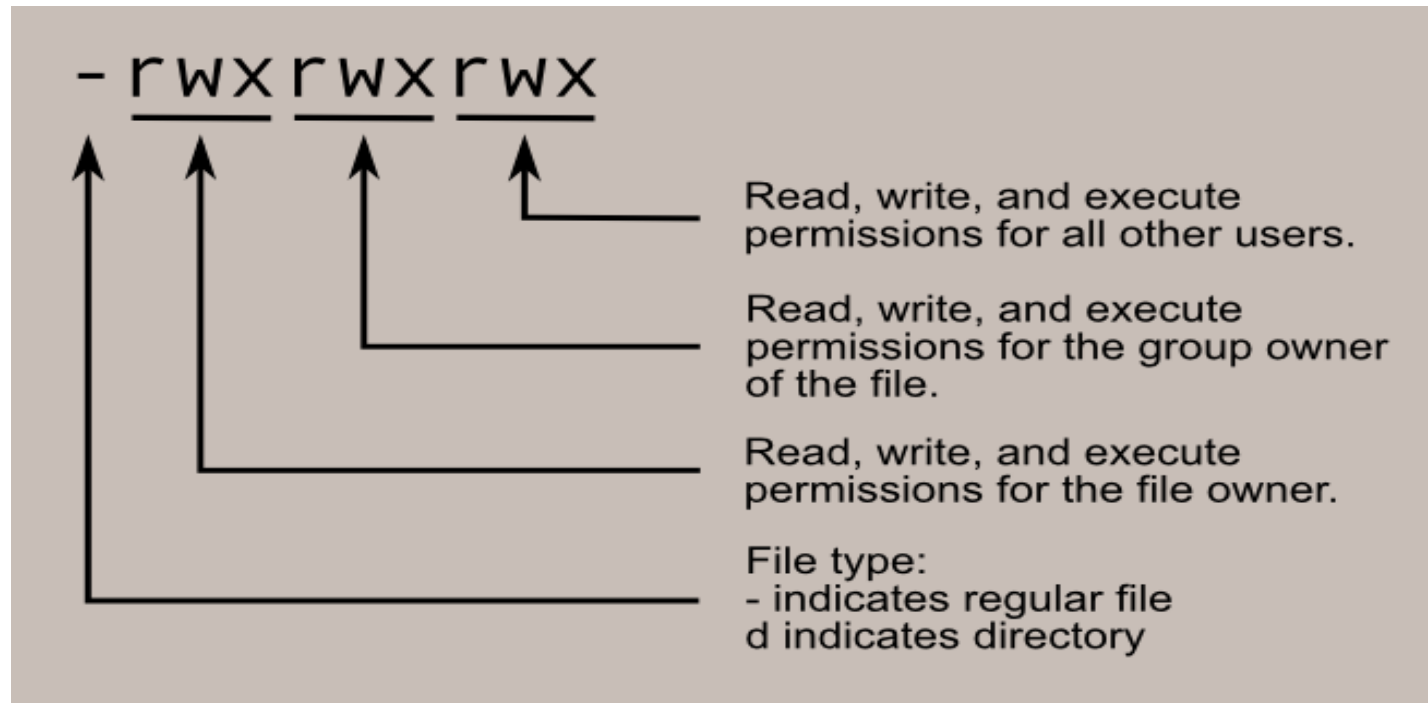
Managing permissions(cont'd)

- To set or modify the permissions for user, groups and others we use **chmod** command. This command can be used in relative or absolute mode.
- Three digits are used to set permissions in absolute mode.

Permission	Numeric Representation
Read	4
Write	2
Execute	1

Managing permissions(cont'd)

- For setting up the permission calculate the value you need.
- Permission format:



Managing permissions(cont'd)

- Example: To set the permission of read, write for user and read for group and read, execute to others for file script.py.

for user: read + write – $4+2 = 6$

for group: read – 4

for others: read + execute – $4 + 1 = 5$

```
[root@localhost test]# chmod 645 script.py
[root@localhost test]# ls -l script.py
-rw-r--r-x. 1 bob test 0 Nov 12 02:18 script.py
```

Regular expressions

- Regular expressions also called as regex is description of pattern of characters. This pattern we define can be used for matching and string parsing at so many instances.
- Regex is the one of the most powerful tool for system administration. It can be used with various utilities in Linux such as grep, egrep, awk etc.
- Regex will search for the pattern matching on each line and returns the appropriate output.

Regular expressions(cont'd)

- Regular expression symbols

Symbol	Name	Description
.	Period	Match exactly single character.
\	Backslash	Turn off the special meaning of the special character.
*	Asterisk	Match item zero or more times.
+	Plus	Match item one or more times.
?	Question mark	Match item zero or more time.
^	Carat	Match the start of the line
\$	Dollar	Match the end of the line

Regular expressions(cont'd)

Symbol	Name	Description
!	Exclamation mark	Do not match any character or regex.
[]	Square brackets	Match any characters in square brackets.
{n}	n enclosed in braces	Match exactly n of the preceding items.
{n,m}	n and m enclosed in braces	Match between n and m of the preceding items.
{n,}	n enclosed in braces	Match n or more of the preceding items.
[^]	Carat in brackets	Match a character which is not one of those in square brackets.
 	Pipe	Match what is on the left or what is on the right of pipe symbol.
()	Parenthesis	Group one ore more regular expression.

Regular expressions(cont'd)

- Best way to learn regex is to take some sample data and work with it creating the expression.
 - Consider the example to match the exact pattern.
expression: a
The bat took a big bite of apple.
 - In the above example it will look for the exact character 'a'.
 - Expression: bat (It will look for the word bat for exact match)
The bat took a big bite of apple.

Regular expressions(cont'd)

- Expression: b.g
The bat took a **big** bite of apple.
- Expression: ^The
The bat took a big bite of apple.
- Expression: \$apple
The bat took a big bite of **apple**.
- Expression: [1-8]
Number: G**4** S**7** C**1** L0 M9

Regular expressions(cont'd)

- Expression: `t[eo]d`

When **today** is over Ted will have a **tedious** time tidying up.

- Expression: `x{5} y{3,}`

xxxxx yyyy

xxxx yyyyyy

Regular expressions(cont'd)

- There are some special character classes that can be helpful:
 - [:digit:] – only the digits 0 to 9
 - [:alnum:] - any alphanumeric character 0 to 9 or A to Z or a to z.
 - [:alpha:] – any alpha character a to z or A to Z.
 - [:blank:] – space and TAB characters only

Regular expressions(cont'd)

- Regex online tools:
 - <https://regex101.com/>
 - <https://regexr.com/>

Regular expressions(cont'd)

Try it out:

- Create file using touch:
 - sample.txt
 - abc.txt
 - regex.txt
 - sum.txt
- list all the files using regex. Use: `ls -l *`
- List all the .txt files. Use: `ls -l *.txt` (It will display all the files with .txt extension)
- List all the files starting with letter s. Use: `ls -l s*` (The files sum.txt and sample.txt will be seen)

grep

- Grep is a multi purpose tool that is used to find specified strings or regular expressions.
- GREP stands for Global Regular Expression Point.
- When the match is found in line for the specified pattern it copies the line to the standard output or the form of output you have requested and displays it.

Grep(cont'd)

- To search for the specific string in the file we use command: **grep** **“specific_string” filename**
 - For example we want to search for string root in /etc/passwd we use: **grep “root” /etc/passwd**

```
[root@localhost ~]# grep "root" /etc/passwd
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
```

- It can be used along with cat command by sending the output to grep using pipe.
example: **cat /etc/passwd | grep “root”**

Grep(cont'd)

- To make the search case insensitive use `-i` option with `grep`. This will match all the
 - Command: `grep -i "string" filename`
 - In this example the user 'mark' and 'MARK' is searched without considering the case.

```
[root@localhost ~]# grep -i "mark" /etc/passwd
mark:x:1005:1005::/home/mark:/bin/bash
MARK:x:1006:1006::/home/MARK:/bin/bash
```

Grep(cont'd)

- Grep used with regular expression to match the string. Command: **grep "regex" filename**
 - We can use various regex which we have seen in the previous section.
 - To match beginning of the line user ^ symbol.

```
[root@localhost ~]# grep "^This" sample.txt  
This is the file that is used for the demo purpose.
```

- This matches the start of the line with the word This.
- Similarly to match end of the line user \$ sign. For example: **grep "This\$" sample.txt**

Grep(cont'd)

- Use `-c` option to count the empty lines with grep. Command: `grep -c "^$"` `filename`.
- To search for the digits between 1 to 8 use command: `grep "[1-8]" filename`
- For match of exactly single character use `.` (dot) except it doesn't match the end of the line character.

example: `grep "i." sample.txt`

```
[root@localhost ~]# grep "i." sample.txt
This is the file that is used for the demo purpose.
Grep can be used with regex.
It matches the pattern in a file.
```

Grep(cont'd)

- Use **-v** option to invert the match. It will display the lines that do not match with the string specified.

command: **grep -v "string" filename**

- To display the line numbers along with the string use **-n** option.

Command: **grep -n "i*" sample.txt**

```
[root@localhost ~]# grep -n "i*" sample.txt
1:This is the file that is used for the demo purpose.
2:Grep can be used with regex.
3:It matches the pattern in a file.
4:And returns the output
```

Grep(cont'd)

- Grep options

Option	description
-i	To ignore the case.
-v	To invert the match.
-c	To print the count of matching lines.
-r	To search recursively for the path provided.
-E	To provide extended regular expression group.
-n	To prefix each line with the line number too.
--color	To print the coloured output.

End of session